Polynomial Regression

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, SGDRegress
or
from sklearn.preprocessing import PolynomialFeatures, Standar
dScaler
from sklearn.metrics import r2_score
from sklearn.pipeline import Pipeline
import plotly.express as px
import plotly.graph_objects as go

# ========== 2D Polynomial Regression ==========
# Generate data
X = 6 * np.random.rand(200, 1) - 3
y = 0.8 * X**2 + 0.9 * X + 2 + np.random.randn(200, 1)

# Plot raw data
plt.figure(figsize=(10, 6))
plt.plot(X, y, 'b.')
plt.xlabel("X")
plt.ylabel("y")
plt.title("Raw Data")
plt.show()

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, tes
t_size=0.2, random_state=2)

# Linear Regression
lr = LinearRegression()
lr.fit(X_train, y_train)
```

```
y_pred = lr.predict(X_test)
print(f"Linear Regression R2 score: {r2_score(y_test, y_pre
d):.4f}")

plt.figure(figsize=(10, 6))
plt.plot(X_train, lr.predict(X_train), color='r', label='Line
ar Regression')
plt.plot(X, y, "b.", alpha=0.5)
plt.xlabel("X")
plt.ylabel("y")
plt.title("Linear Regression Fit")
plt.legend()
plt.show()

# Polynomial Regression (degree=2)
poly = PolynomialFeatures(degree=2, include_bias=True)
X_train_trans = poly.fit_transform(X_train)
X_test_trans = poly.transform(X_test)

lr_poly = LinearRegression()
lr_poly.fit(X_train_trans, y_train)
y_pred_poly = lr_poly.predict(X_test_trans)
print(f"Polynomial Regression R2 score: {r2_score(y_test, y_p
red_poly):.4f}")
print("Coefficients:", lr_poly.coef_)
print("Intercept:", lr_poly.intercept_)

# Plot polynomial fit
X_new = np.linspace(-3, 3, 200).reshape(200, 1)
X_new_poly = poly.transform(X_new)
y_new = lr_poly.predict(X_new_poly)

plt.figure(figsize=(10, 6))
plt.plot(X_new, y_new, "r-", linewidth=2, label="Polynomial F
it")
plt.plot(X_train, y_train, "b.", label='Training points')
```

```
plt.plot(X_test, y_test, "g.", label='Testing points')
plt.xlabel("X")
plt.ylabel("y")
plt.title("Polynomial Regression (degree=2)")
plt.legend()
plt.show()

# SGD with Polynomial Features
sgd = SGDRegressor(max_iter=100)
sgd.fit(X_train_trans, y_train)
y_pred_sgd = sgd.predict(X_test_trans)

plt.figure(figsize=(10, 6))
plt.plot(X_new, sgd.predict(poly.transform(X_new)), "r-", lin
ewidth=2,
         label=f"SGD Predictions (R2={r2_score(y_test, y_pred
_sgd):.2f})")
plt.plot(X_train, y_train, "b.", label='Training points')
plt.plot(X_test, y_test, "g.", label='Testing points')
plt.xlabel("X")
plt.ylabel("y")
plt.title("SGD with Polynomial Features")
plt.legend()
plt.show()

# ========== 3D Polynomial Regression ==========
# Generate 3D data
x = 7 * np.random.rand(100, 1) - 2.8
y = 7 * np.random.rand(100, 1) - 2.8
z = x**2 + y**2 + 0.2*x + 0.2*y + 0.1*x*y + 2 + np.random.ran
dn(100, 1)

# 3D Scatter plot
fig = px.scatter_3d(x=x.ravel(), y=y.ravel(), z=z.ravel())
fig.update_layout(title="3D Data Points", scene=dict(xaxis_ti
tle='X', yaxis_title='Y', zaxis_title='Z'))
```

```
fig.show()

# Prepare grid for surface plot
x_input = np.linspace(x.min(), x.max(), 10)
y_input = np.linspace(y.min(), y.max(), 10)
xGrid, yGrid = np.meshgrid(x_input, y_input)
final = np.vstack((xGrid.ravel().reshape(1,100), yGrid.ravel
().reshape(1,100))).T

# Linear Regression in 3D
lr_3d = LinearRegression()
lr_3d.fit(np.hstack((x, y)), z)
z_final = lr_3d.predict(final).reshape(10, 10)

fig = px.scatter_3d(x=x.ravel(), y=y.ravel(), z=z.ravel())
fig.add_trace(go.Surface(x=x_input, y=y_input, z=z_final, opa
city=0.7))
fig.update_layout(title="Linear Regression Fit in 3D")
fig.show()

# Polynomial Regression in 3D (degree=2)
poly_3d = PolynomialFeatures(degree=2)
X_multi = np.hstack((x, y))
X_multi_trans = poly_3d.fit_transform(X_multi)

lr_poly_3d = LinearRegression()
lr_poly_3d.fit(X_multi_trans, z)

X_test_multi = poly_3d.transform(final)
z_final_poly = lr_poly_3d.predict(X_test_multi).reshape(10, 1
0)

fig = px.scatter_3d(x=x.ravel(), y=y.ravel(), z=z.ravel())
fig.add_trace(go.Surface(x=x_input, y=y_input, z=z_final_pol
y, opacity=0.7))
fig.update_layout(title="Polynomial Regression (degree=2) in
```

```
3D")
fig.show()
```